

Anotações curso em vídeo JS

Recursividade em funções no JavaScript é quando uma função chama ela mesma repetidamente para resolver um problema em partes menores até atingir uma condição de parada.

Módulo A - Conhecendo a linguagem

Aula 1



JavaScript é client side. Opera no lado do cliente

Sendo assim, quando você utiliza o console do desenvolvedor e faz alterações na página, está alterando somente aquela cópia que chegou até você.

Aula 2



NodeJS nada mais é que uma máquina que roda JavaScript fora do navegador

Aula 3



Recomendação de livros:

- JavaScript O guia definitivo
- JavaScript Guia do Programador
- Mozilla guia de referencia: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide>
- Ecma guia de referencia: <https://ecma-international.org/>

Aula 4

Estruturação das pastas para aulas, configuração de ambiente e primeiros passos com html, css e js.

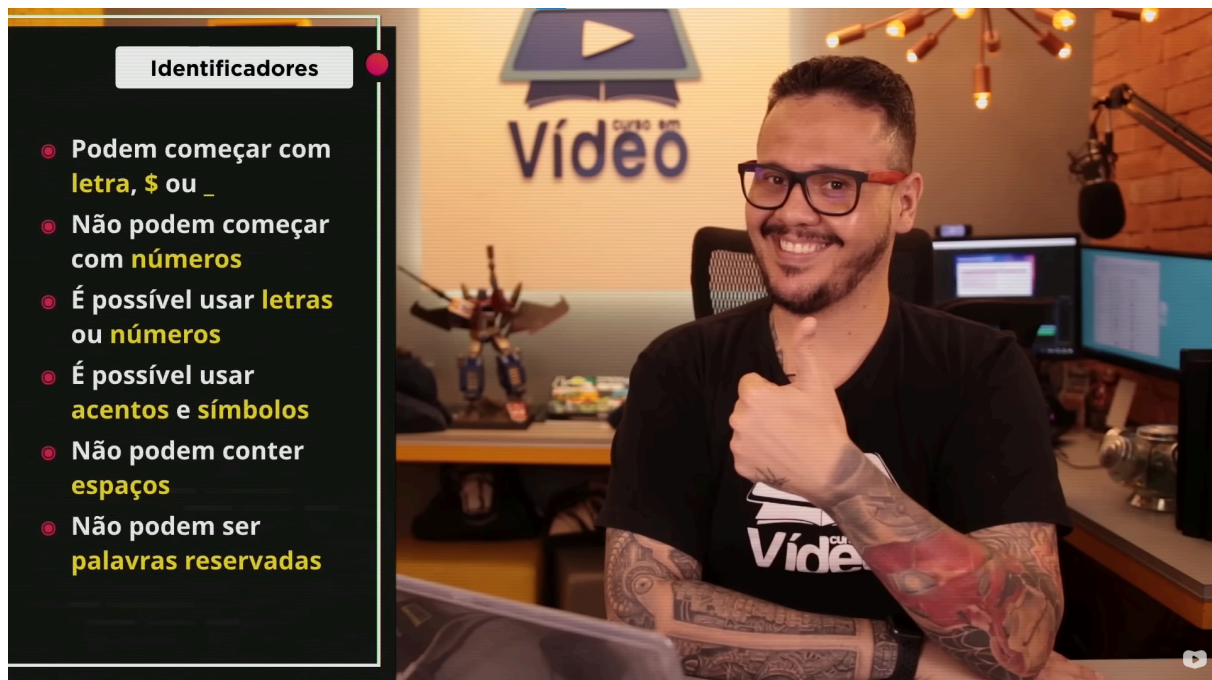
Módulo B - Comandos básicos

Aula 5

Comentários em js:

- //
- /* */

Regras básicas para criação de variáveis (alocações de espaços na memória)



Exemplo básico de utilização typeof

```
let n = 20
undefined
typeof n
'number'
n = "livro"
'livro'
```

```
typeof n  
'string'
```

Aula 6

Exemplo interessante de manipulação de tipos de dados numéricos

```
var n1 = Number(window.prompt('Digite um número: ')) // String originalmente  
var n2 = Number(window.prompt('Digite outro número'))  
// String originalmente  
  
var soma = n1 + n2  
window.alert(`A soma dos valores é ${soma}`) // Concatenação  
// (number + number) para adição  
// (string + string) para concatenação
```



Caso seja necessário forçar um tipo específico de number, pode se usar:

```
var n = Number.parseInt || var n = Number.parseFloat
```

String



Em caso de Strings:

```
String(n) || n.toString()
```



template strings (as famosas crases ``) convertem automaticamente para string

Anotações adicionais para formatação de strings

```
// Formatando Strings
var s = 'JavaScript'
s.length           // quantos caracteres a string tem
s.toUpperCase()    // tudo para 'MAIÚSCULAS'
s.toLowerCase()    // tudo para 'minúsculas'
```

Exemplo de código utilizando essas funcionalidades

```
<script>
  var nome = window.prompt('Qual é seu nome?')
  document.write(`Olá, <strong>${nome}</strong>! Seu nome
tem ${nome.length} letras<br>`)
  document.write(`Seu nome em maiúsculas é ${nome.toUpperCase()}<br>`)
  document.write(`Seu nome em minúsculas é ${nome.toLowerCase()}`)
</script>
```

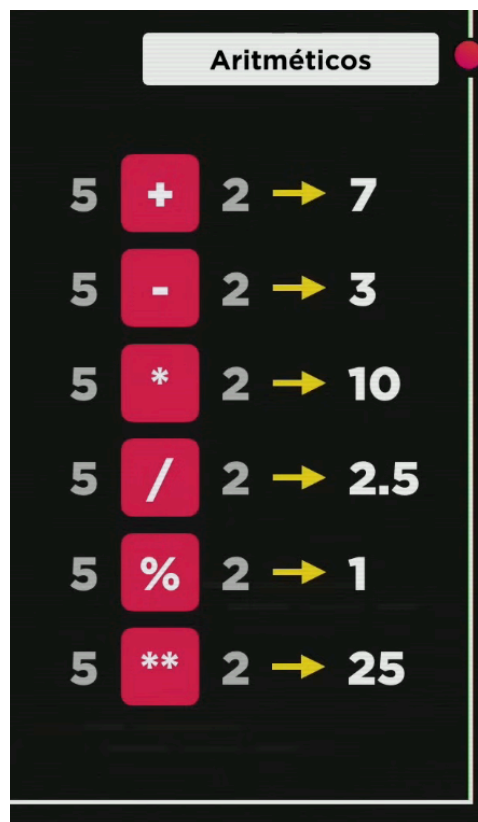
Exemplo interessante formatação de números

```
var n1 = 1545.5
undefined
n1
1545.5
n1.toFixed(2).replace('.', ',')
'1545,50'
n1.toLocaleString('pt-BR', {style: 'currency', currency: 'BRL'})
'R$ 1.545,50'
n1.toLocaleString('pt-BR', {style: 'currency', currency: 'U
```

```
SD'}})  
'US$ 1.545,50'  
n1.toLocaleString('pt-BR', {style: 'currency', currency: 'EUR'})  
'€ 1.545,50'
```

Aula 7

Operadores Aritméticos



Ordem de Precedência

Atribuição Simples

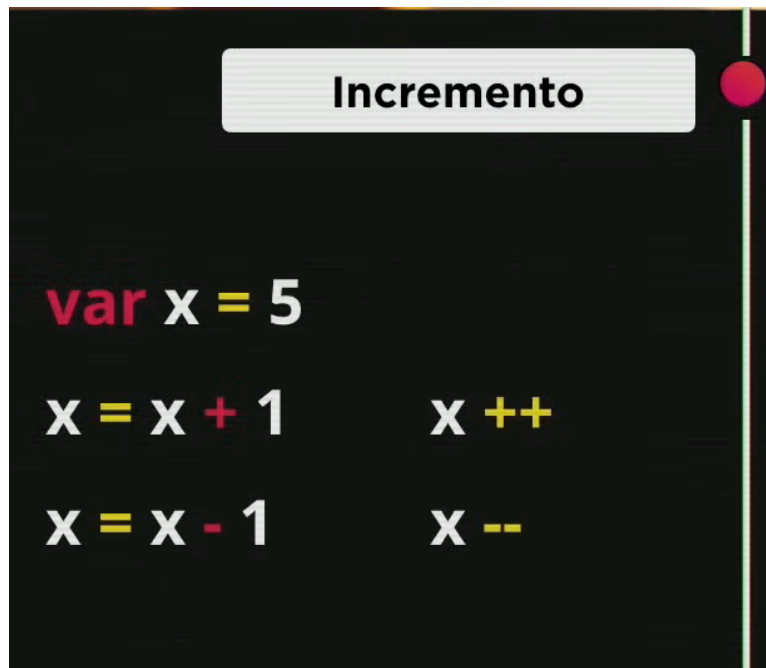
<code>var a = 5 + 3</code>	8
<code>var b = a % 5</code>	3
<code>var c = 5 * b ** 2</code>	45
<code>var d = 10 - a / 2</code>	6
<code>var e = 6 * 2 / d</code>	2
<code>var f = b % e + 4 / e</code>	3

Auto atribuição

Simplificando

<code>var n = 3</code>	
<code>n = n + 4</code>	<code>n += 4</code>
<code>n = n - 5</code>	<code>n -= 5</code>
<code>n = n * 4</code>	<code>n *= 4</code>
<code>n = n / 2</code>	<code>n /= 2</code>
<code>n = n ** 2</code>	<code>n **= 2</code>
<code>n = n % 5</code>	<code>n %= 5</code>

Operadores de incremento simplificados



Exemplo de código

```
var n = 10  
undefined  
n  
10  
n++  
10  
n  
11  
n--  
11  
n  
10  
++n  
11  
--n  
10
```

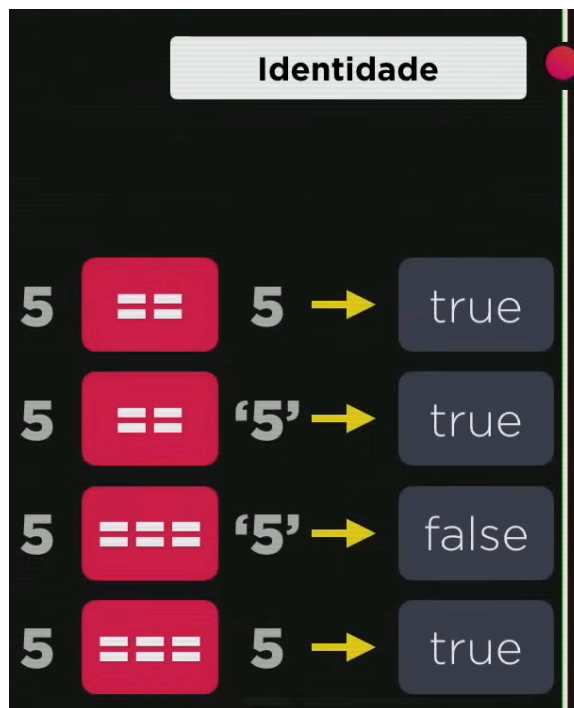
Aula 8

Operadores relacionais

Relacionais				
5	>	2	→	true
7	<	4	→	false
8	>=	8	→	true
9	<=	7	→	false
5	==	5	→	true
4	!=	4	→	false

// Exemplos	
preço >= 200.50	// o preço é maior ou igual a 200.50?
idade < 18	// a idade é menor do que 18?
curso == 'JavaScript'	// o curso é JavaScript?
n1 != n2	// o primeiro número é diferente do segundo?

Operadores de identidade

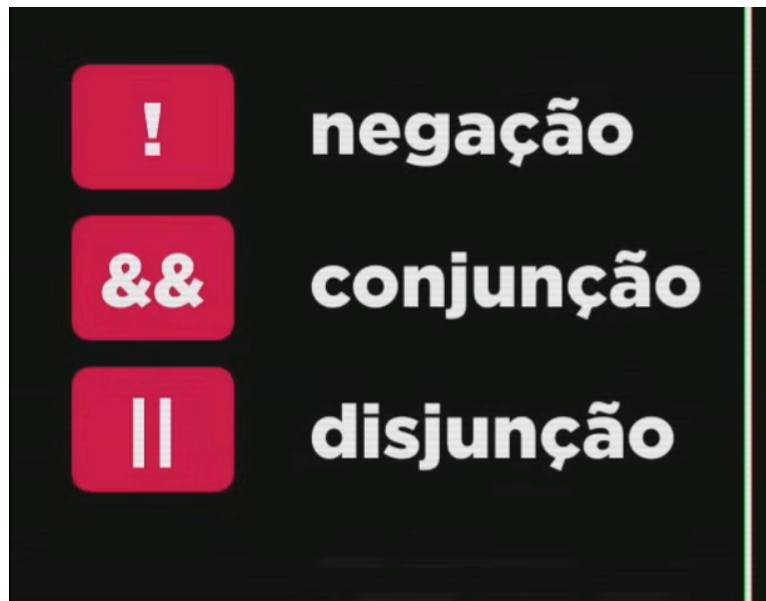


Exemplo

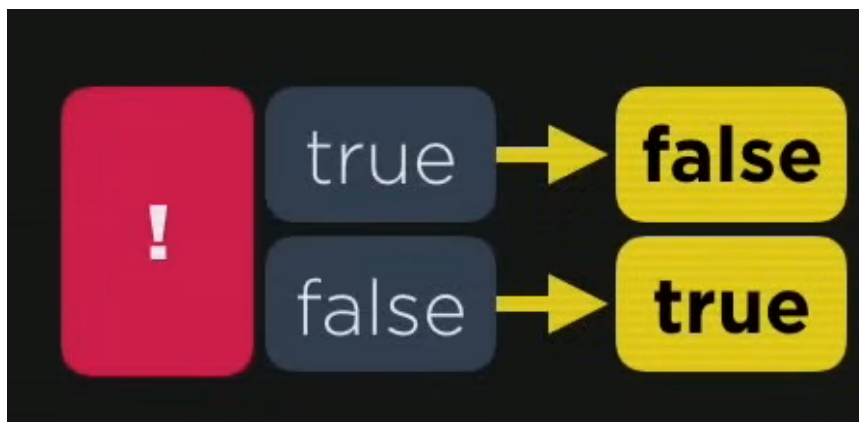
```
5 == '5'  
true  
var x = 5  
undefined  
var y = '5'  
undefined  
typeof x  
'number'  
typeof y  
'string'  
x == y  
true  
x === y  
false  
> x != y  
false  
> x !== y  
true
```

Operadores Lógicos

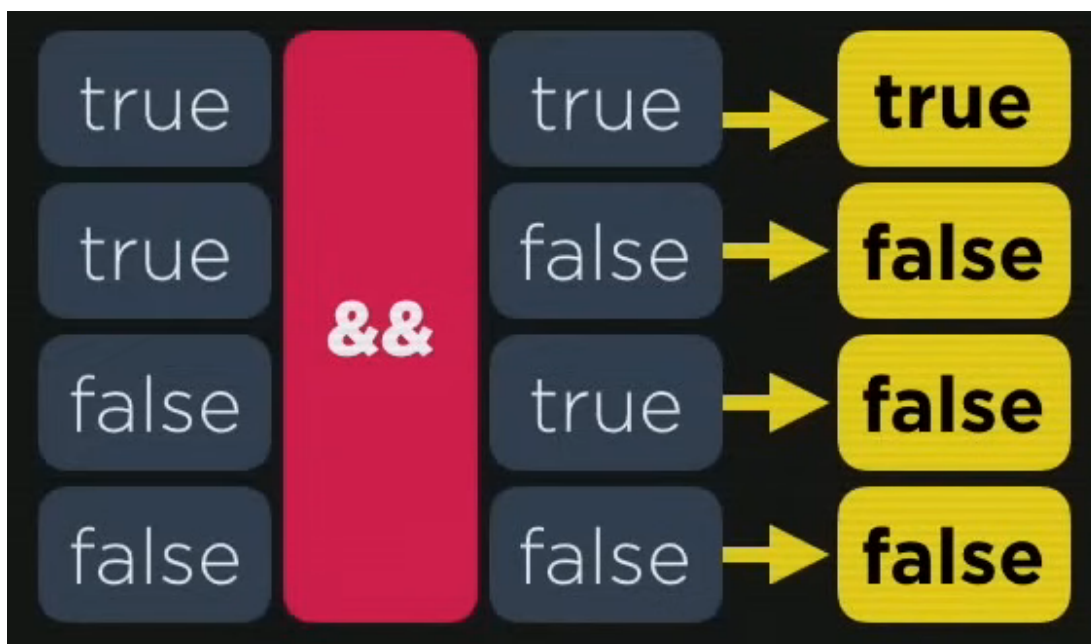
Tipos



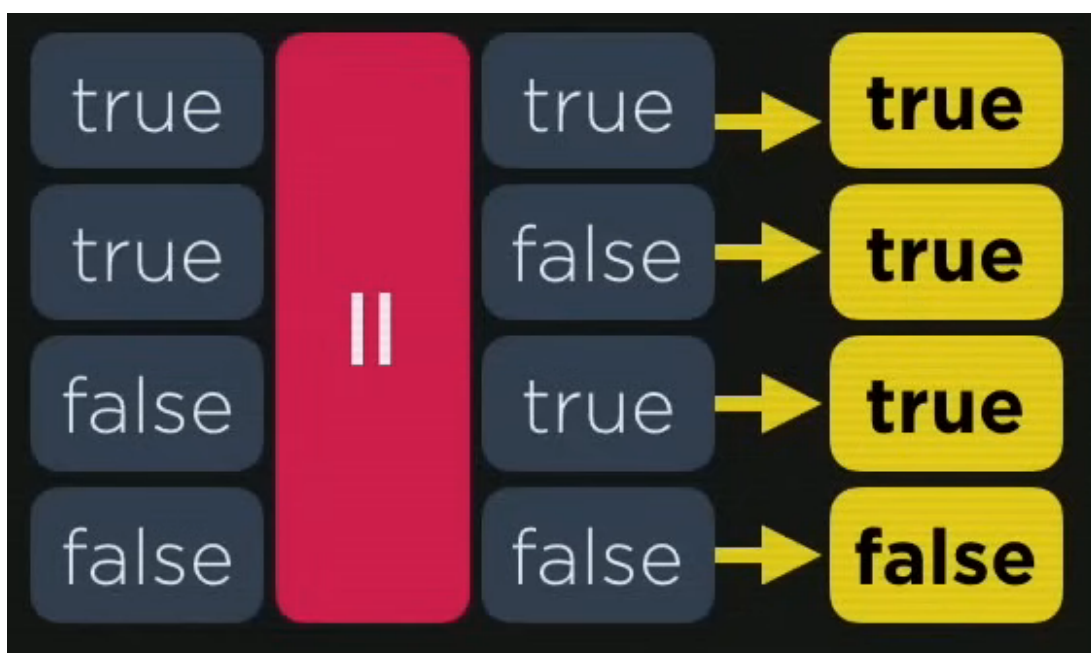
Negação



Conjunção

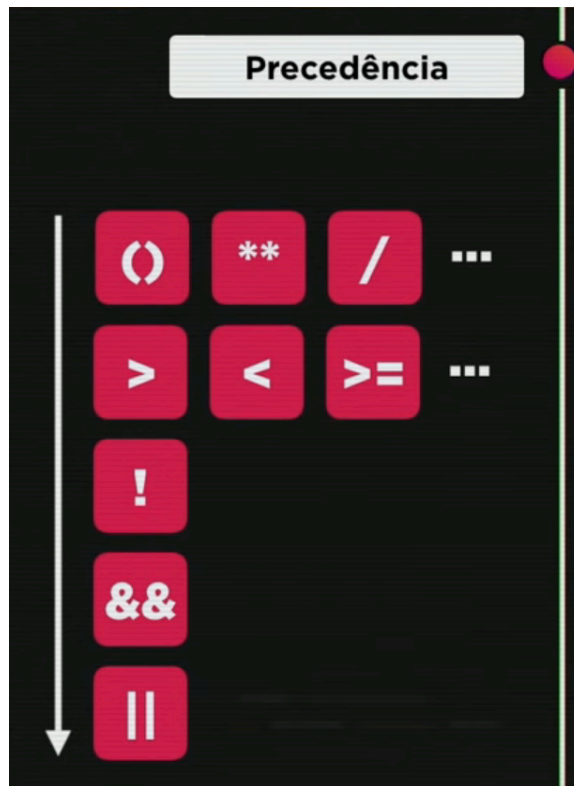


Disjunção

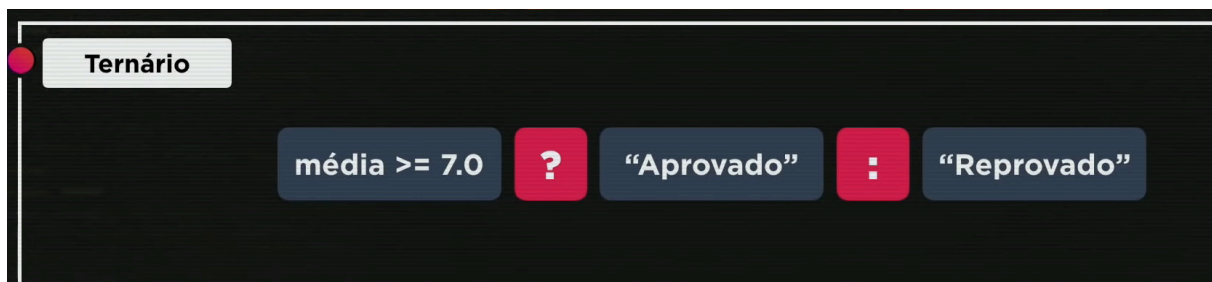
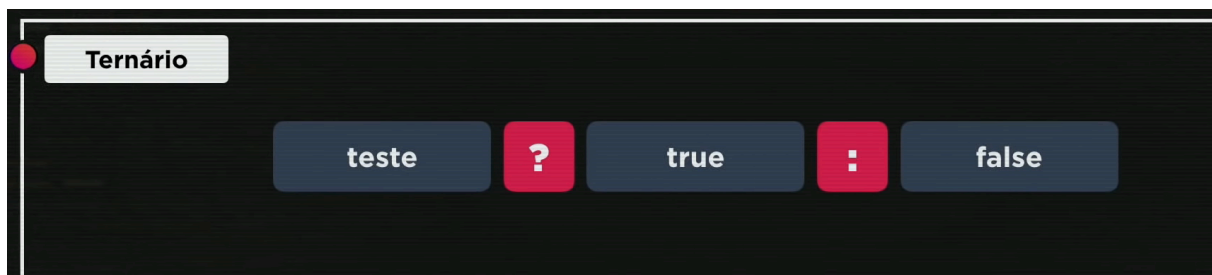


Em caso de uma longa expressão com os 3 tipos de operadores, a lógica de execução vai ser: Primeiro o NÃO, depois o E e por último o OU.

Ordem de Precedência Geral



Operador Ternário



Exemplos

```
> var média = 5.5  
undefined  
> média > 7 ? 'APROVADO' : 'REPROVADO'  
'REPROVADO'
```

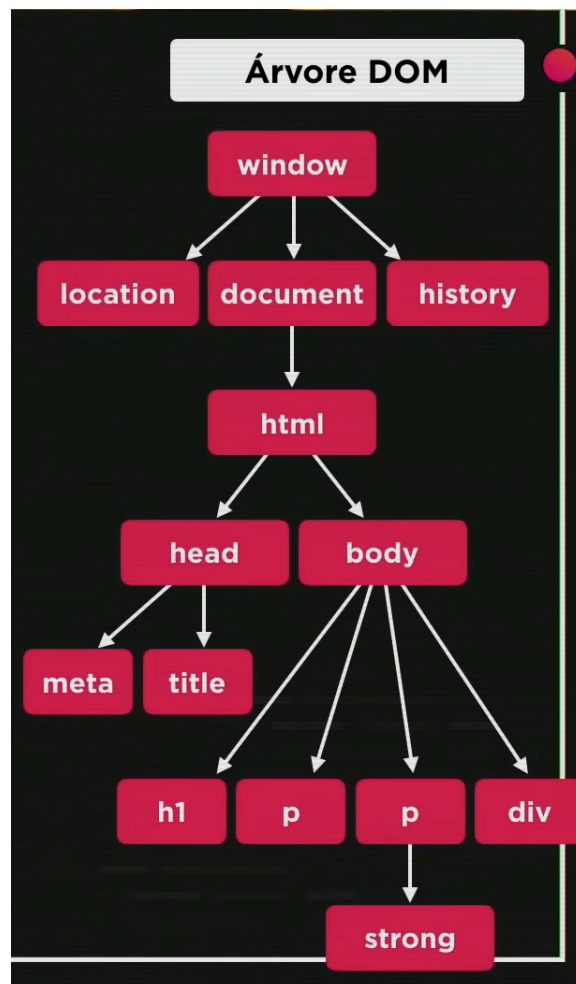
```
> média += 3
8.5
> média
8.5
> média > 7 ? 'APROVADO' : 'REPROVADO'
'APROVADO'
```

```
> var x = 8
undefined
> var res = x % 2 == 0 ? 5 : 9
undefined
> x
8
> res
5
> var idade = 19
undefined
> var r = idade >= 18 ? 'MAIOR' : 'MENOR'
undefined
> r
'MAIOR'
```

Módulo C - Entendendo o DOM (Document Object Model)

Aula 9

Exemplo de árvore hierárquica dom



InnerText pega somente o texto do elemento selecionado, já o innerHTML pega toda a formatação, inclusive as tags filhas

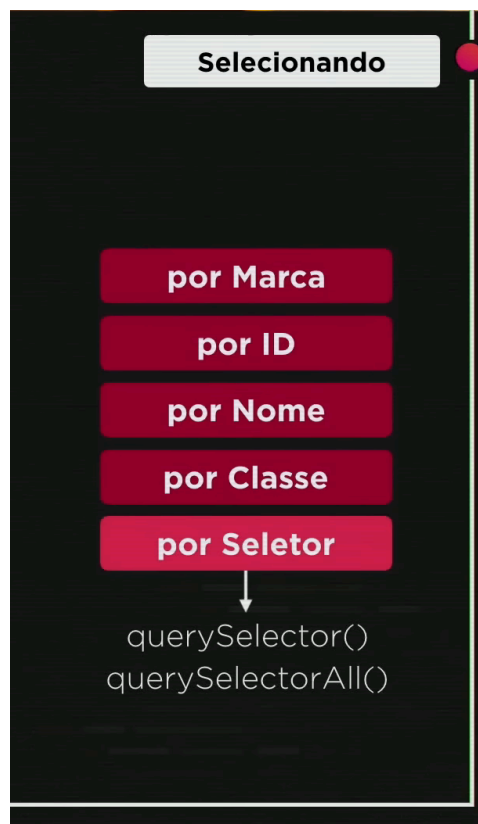
Exemplo de utilização `getElementsByTagName()`

```
<body>
  <h1>Iniciando estudos DOM</h1>
  <p>Aqui vai o resultado</p>
  <p>Aprendendo a usar <strong>DOM</strong> em JavaScript</p>
  <div>Clique em mim</div>
  <script>
    var p1 = window.document.getElementsByTagName('p')[0]
    // 0 0 indica que eu quero mudar um elemento específico
    // o, no caso o primeiro, funciona assim como nos arrays
    //window.document.write('Está escrito assim: ' + p1.inn
```

```
erText)  
  p1.style.color = 'blue'  
</script>
```



Não coloquei outros exemplos pq o princípio é o mesmo para todas as seleções



Exemplo query selector

```
var d = window.document.querySelector('div.msg') //a estrutura de seleção com o ponto se deve pelo fato de o elemento estar sendo identificado por classe, se fosse id seria ('div#msg')  
d.style.background = 'blue'
```

Aula 10

| Eventos DOM são coisas que eu posso fazer com um elemento



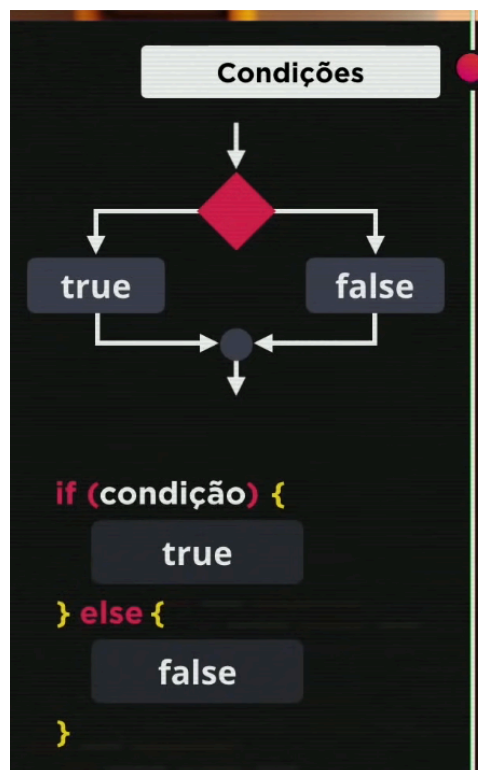
Lista de eventos DOM em js: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Events

Módulo D - Condições em JavaScript

Aula 11



Abaixo temos de maneira bem interessante e ilustra a representação de blocos condicionais



Aula 12

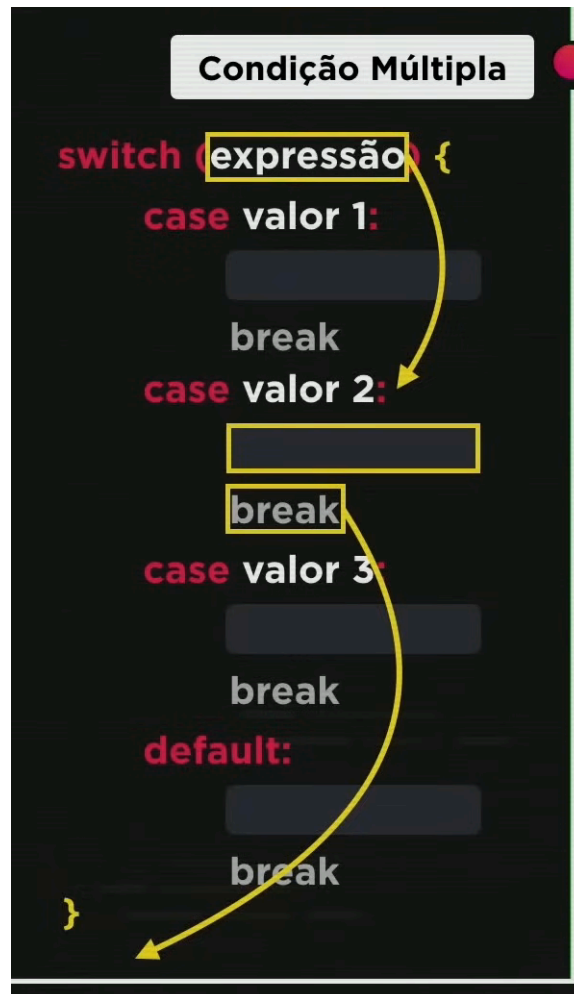
Exemplo interessante de blocos condicionais

```
var idade = 22  
console.log(`Você tem ${idade} anos.`)  
if(idade < 16) {  
  console.log(`Não vota`)
```



```
} else if(idade < 18 || idade > 65) {  
  console.log(`Voto opcional`)  
} else {  
  console.log(`Voto obrigatório`)  
}
```

Estrutura switch case



O break é fundamental na estrutura do switch

Módulo E - Repetições

Aula 13

Exemplo de código while/do while

```
var c = 1

do{
  console.log('Tudo bem?')
  c++
} while (c <=6)

/*while (c <=6) {
  console.log('Tudo bem?')
  c++
}*/

/*
console.log('Tudo bem?')
console.log('Tudo bem?')
console.log('Tudo bem?')
console.log('Tudo bem?')
console.log('Tudo bem?')
console.log('Tudo bem?')
*/
```



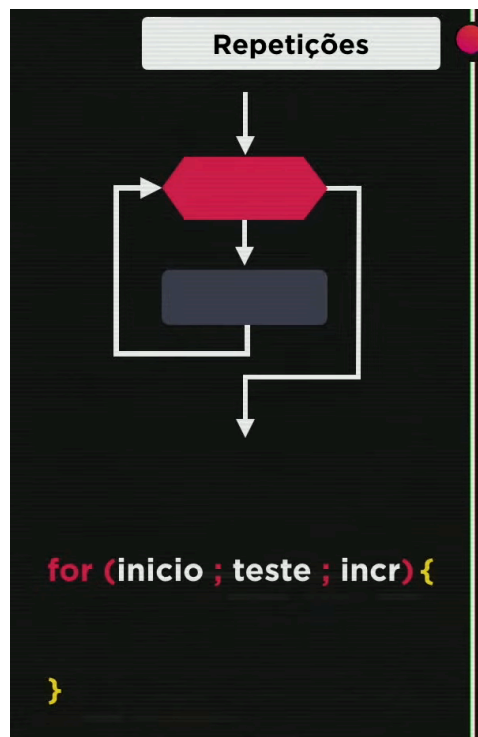
Ambas as estruturas acima realizam a mesma função:

while = faça enquanto

do while = enquanto faça

Aula 14

Estrutura do for

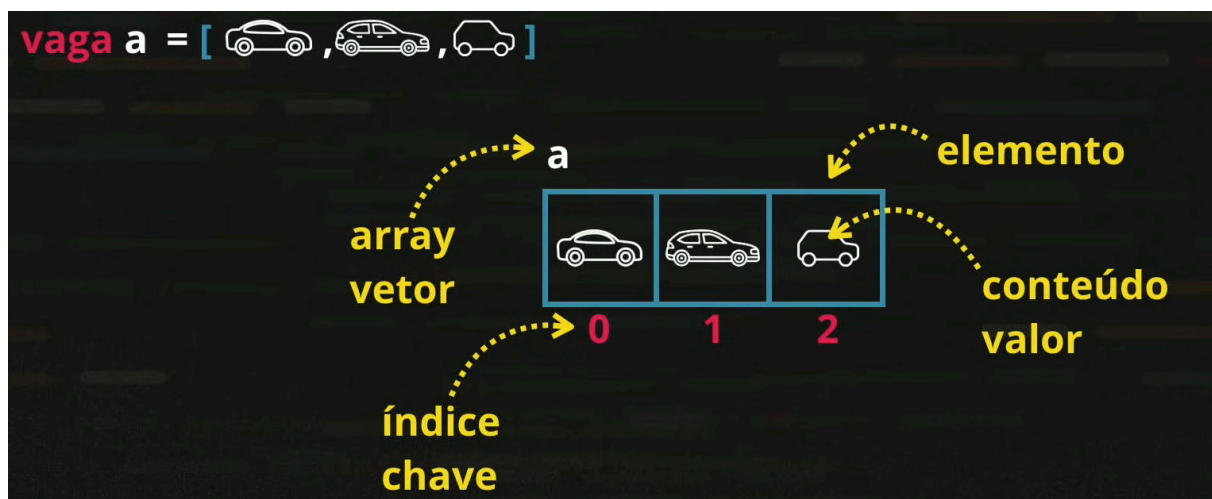


Módulo F - Arrays e Funções

Aula 15



Um array ou variável composta é uma variável que contém vários elementos e cada elemento é composto pelo seu valor e uma chave de identificação



Exemplos

```
let num = [ 5 , 8 , 4 ]
```

```
num[3] = 6
```

```
num.push(7)
```

num

5	8	4	6	7
0	1	2	3	4

```
for(let pos=0; pos<num.length; pos++) {  
  console.log(num[pos])  
}
```

```
for(let pos in num) {  
  console.log(num[pos])  
}
```

num

4	5	6	7	8
0	1	2	3	4



Quando o método `indexOf` não encontra o valor especificado ele retorna `-1`

Aula 16

Exemplo de função básica

```
function parimp(n) {  
  if (n%2==0) {  
    return 'par'  
  } else {  
    return 'ímpar'  
  }  
}  
  
let res = parimp(11)
```

Diagram annotations:

- A dashed blue arrow labeled "parâmetro" points to the parameter `n` in the function definition.
- A dashed blue arrow labeled "ação" points to the `return` statement inside the `if` block.
- A dashed blue arrow labeled "chamada" points to the function call `parimp(11)` in the `let` statement.



Recursividade em funções no JavaScript é quando uma função chama ela mesma repetidamente para resolver um problema em partes menores até atingir uma condição de parada.

Aula 17

Diferença Arrays x Obejtos

array

```
let num = [ 5 , 8 , 4 ]
```

num

5	8	4
0	1	2

object

```
let amigo = {nome:'José', sexo:'M', peso:85.4, engordar(p){}}
```

amigo

José	M	85.4	[function]
nome	sexo	peso	engordar()

